

# Proposta Executiva Solução Big Data para uma Frota Autônoma Conectada

Arquitetura de dados resiliente, escalável e em tempo real, orientada a valor de negócio.

FOLHA DE ROSTO • COMPONENTES DO GRUPO (NOME COMPLETO + DRE)

Izabela Lima da Silva

DRE 124156557

Caio Meirelles

DRE 122071557

Esta proposta especifica uma solução de **Big Data** para monitorar, em tempo real, uma frota de veículos autônomos e conectados de um consórcio de seis locadoras. O documento é orientado a **decisão e valor de negócio**: apresenta o problema, o retorno esperado, a arquitetura e a justificativa objetiva de cada escolha. A fundamentação teórica aprofundada e as referências estão consolidadas no *relatório acadêmico* que acompanha o trabalho.

#### DIVISÃO DO TRABALHO (DEFESA ORAL)

**Parte A — Infraestrutura & fluxo** (borda, ingestão, arquitetura Kappa, processamento, robustez): seções 3–5. **Parte B — Armazenamento, interface & IA** (persistência poliglota, dashboard, cobrança, concierge): seções 6–7. Emergências (hot path) é a ponte.

### Sumário

- |                                                            |                                                 |
|------------------------------------------------------------|-------------------------------------------------|
| 1. O problema e o valor de negócio                         | 6. Serving: dashboard, cobrança e emergências   |
| 2. Do DW (Avaliação 02) ao Big Data e a extensão do modelo | 7. Concierge de viagem por IA                   |
| 3. Visão da solução (pipeline)                             | 8. Casos de uso e rastreabilidade das entidades |
| 4. Decisões de arquitetura e justificativas                | 9. Custos, nuvem e prova de conceito            |
| 5. Robustez: tratamento de casos limítrofes                | 10. Alternativas descartadas                    |
|                                                            | 11. Conclusão                                   |

## 01 O problema e o valor de negócio

O consórcio automatizou a operação: pelo aplicativo, o cliente reserva e indica o local; o **veículo sai sozinho do pátio**, navega no trânsito e estaciona no ponto pedido; ao fim, retorna para higienização. Cada veículo é um **sensor sobre rodas** (motor, chassi, câmeras 360°, impacto) com computador de bordo. O sistema deve sustentar quatro necessidades — acompanhamento, cobrança automática, apoio a emergências e um concierge por IA — sob banda móvel **cara e finita**, alta concorrência **sem perda de pacotes** e **ordenação estrita por veículo e timestamp**.

A arquitetura é meio; o valor é o fim. Cada capacidade técnica move um indicador do negócio:



## 02 Do DW (Avaliação 02) ao Big Data e a extensão do modelo

Na etapa anterior modelamos um **Data Warehouse** dimensional em PostgreSQL — esquema estrela com os fatos **Reserva**, **Locação** e **Movimentação** sobre dimensões conformadas, integrando 4 fontes OLTP, com previsão de ocupação de pátios por **cadeias de Markov**. Esse modelo é batch (D-1) e não cobre a frota conectada.

Em vez de manter DW e data lake separados, adotamos o **Lakehouse**: o esquema estrela passa a ser a camada **Gold**, alimentada por ELT a partir do streaming. Estendemos o modelo com quatro novos fatos (**Telemetria**, **Cobrança**, **Sinistro**, **Manutenção**) e novas dimensões (**Tempo\_Detalhe** de grão minuto, **Sensor** SCD-2, **TipoEvento**, **FaixaCondução**). A **Fato\_Cobranca** dá **continuidade** ao pagamento já modelado na Avaliação 02 — a entidade **Pagamento** (com **valor**) do OLTP da Parte I, esboçada como *Fato\_Pagamento*: traz esse valor para a camada analítica (Gold), no grão de uma locação faturada, e o enriquece com os ajustes dinâmicos pós-uso (km, consumo, infrações, score de condução) que a frota conectada passa a exigir. O DDL da extensão ( **06\_ddl\_extensao.sql** ) foi **validado** em PostgreSQL. (Detalhamento na seção 4 do relatório acadêmico.)

### 03 Visão da solução PARTE A

O pipeline cobre ingestão → processamento → armazenamento → visualização, com três caminhos de latências distintas.

```
# Pipeline fim a fim
[Borda/Edge (Avro)] > [MQTT > ponte > Kafka (telemetry.raw | alerts.critical)] >
  └─ HOT colisão → Spark Streaming → despacho + dossiê
  └─ WARM telemetria 1s → Flink (event-time/watermarks) → rotas, score
  └─ COLD Delta Bronze → Spark batch → km, manutenção, Markov
[Delta Lake Bronze→Silver→Gold (DW estendido)] > [NoSQL: Cassandra · Mongo · Redis]
> [Streamlit · Cobrança · Concierge]
```

A arquitetura de referência acima **detalha e amplia o pipeline-base do grupo** (Sensores → Kafka → Flink → NoSQL → Dashboard), acrescentando os regimes Spark (lote histórico e *hot path* de emergências) e o Lakehouse Delta, que garante ACID entre lote e streaming.

### 04 Decisões de arquitetura e justificativas PARTE A

Cada camada resolve um requisito não funcional específico. As justificativas abaixo são objetivas; o embasamento teórico completo está no relatório acadêmico.

| Decisão                  | Por quê (justificativa)                                                                                                                                                                                                                                                 | Descartamos            |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| Borda filtra (Avro/MQTT) | Banda é o recurso escasso; só agregados e exceções trafegam. A reação sub-segundo à colisão é embarcada. (Dean & Ghemawat 2004)                                                                                                                                         | Enviar vídeo bruto     |
| Kafka (log particionado) | Tópicos isolados (telemetria vs. alarme); partição por <code>vehicle_id</code> dá ordem por veículo; replay e durabilidade sob concorrência. (Kreps 2011)                                                                                                               | HTTP síncrono no banco |
| Kappa + Delta Lake       | Um só código; o Delta traz ACID e time travel ao object store, unificando lote e streaming. (Fragkoulis 2020; Armbrust 2020)                                                                                                                                            | Lambda (código duplo)  |
| 3 regimes (Spark/Flink)  | SLAs distintos: vazão histórica (Spark batch, que <b>generaliza o MapReduce</b> — map/reduce dá paralelismo e tolerância por re-execução), emergência (Spark Streaming) e latência mínima com watermarks (Flink). (Dean & Ghemawat 2004; Zaharia 2012/13; Carbone 2015) | Um motor único         |
| Persistência poliglota   | Teorema CAP: cada dado no banco certo — Cassandra (telemetria/AP), Mongo (cadastrais), Redis (cache), PostgreSQL/Delta (cobrança/CP). (Gilbert & Lynch 2002)                                                                                                            | Banco relacional único |

Túneis e zonas de sombra entregam dados fora de ordem. A solução combina compressão e *store-and-forward* na borda, ordem por `vehicle_id` no Kafka e **event-time + watermarks** no Flink, garantindo a ordenação estrita por veículo e timestamp mesmo sob rede oscilante.

## 05 Robustez: tratamento de casos limítrofes PARTE A

O enunciado exige operar sob **banda finita, alta concorrência sem perda de pacotes e ordenação estrita por veículo e timestamp mesmo sob forte estresse**. A tabela mapeia cada situação-limite ao mecanismo que a trata: nenhum dado é perdido e nenhum resultado fica inconsistente.

| Situação-limite                                          | Como o sistema lida                                                                                                                                                                                                                                                      |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Túnel / zona de sombra (queda de conexão)                | A borda faz <i>store-and-forward</i> (buffer local persistente) e drena em ordem ao reconectar. Colisão sem sinal: <i>ring buffer</i> local (~30 s) e o <i>payload</i> de emergência prioriza a rede assim que ela volta. <b>Zero perda.</b>                             |
| Dados fora de ordem / atrasados                          | Ordem por <code>vehicle_id</code> na partição do Kafka + <b>event-time e watermarks</b> no Flink reordenam. Eventos além da tolerância ( <i>allowed lateness</i> ) vão para <i>side output</i> e são reconciliados no lote sobre o Lakehouse — <b>nunca descartados.</b> |
| Relógio do veículo dessincronizado ( <i>clock skew</i> ) | Timestamp carimbado na borda; processamento por <i>event-time</i> ; as watermarks toleram o desvio e a conformação no Silver normaliza o eixo temporal.                                                                                                                  |
| Duplicatas e reproprocessamento ( <i>replay</i> )        | Idempotência por chave ( <code>vehicle_id+timestamp ; id_locacao</code> na cobrança). A cobrança é <b>exactly-once</b> (transação do Delta) — jamais cobra em dobro, mesmo com reenvio.                                                                                  |
| Pico de carga / surto (tempestade, evento na cidade)     | O Kafka absorve como <i>buffer</i> durável; o particionamento escala consumidores e o <i>autoscaling</i> acompanha; <i>backpressure</i> no Flink evita colapso. O tópico <code>alerts.critical</code> é isolado — o alarme nunca fica preso atrás da telemetria.         |
| Falha de nó ou <i>broker</i>                             | Replicação/ISR no Kafka; o RDD recompõe partições por <b>linhagem</b> ; o Delta garante ACID (sem estado corrompido); serviços <i>stateless</i> reescalonom.                                                                                                             |
| Sensor defeituoso / firmware divergente                  | O <i>Schema Registry</i> valida o Avro; mensagens malformadas vão para <i>dead-letter</i> (o Bronze guarda o bruto para reprocesso). A <code>Dim_Sensor</code> (SCD-2) sabe qual firmware rodava; leituras implausíveis são marcadas.                                    |

| Situação-limite                                                 | Como o sistema lida                                                                                                                                                                      |
|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Evento tardio após fechar a fatura</b> (cobrança em disputa) | O <i>time travel</i> do Delta + janela de conciliação permitem um ajuste posterior <b>auditável</b> (nova versão da cobrança) sem reescrever o histórico.                                |
| <b>Partição de rede</b> (CAP)                                   | A telemetria (AP) segue aceitando escrita (consistência eventual); a cobrança e o dossiê (CP) preferem consistência e aguardam a reconciliação — evitando fatura ou laudo inconsistente. |
| <b>Emergências simultâneas em massa</b>                         | O <i>hot path</i> prioriza <code>alerts.critical</code> e escala horizontalmente; o SLA de resposta é preservado mesmo com múltiplos sinistros concorrentes.                             |
| <b>Concierge: pergunta ambígua ou sem dado</b>                  | O RAG responde com <b>proveniência</b> (cita a fonte) e, na dúvida, pede esclarecimento em vez de alucinar; sem POI adequado, oferece alternativa. A base vetorial é versionada.         |

#### GARANTIA DE PROJETO

Todo caminho de dado tem resposta para a falha: **durabilidade** (Kafka/Delta/store-and-forward), **ordem** (partição + watermarks), **idempotência** (dedup/exactly-once) e **reconciliação** (Lakehouse + time travel). É o que sustenta "sem perda de pacotes" e "ordenação estrita sob estresse".

## 06 Serving: dashboard, cobrança e emergências PARTE B

### Dashboard (Streamlit)

Painel OLAP sobre a camada **Gold** (nunca sobre os OLTP), atualizado em quase tempo real via Redis. Traz mapa geoespacial com **trânsito, engarrafamentos, meteorologia e eventos/alertas da cidade** (fontes externas que afetam a circulação da frota), cartões consolidados (veículos ativos, bateria média, alertas mecânicos), **triagem de incidentes que pisca em vermelho**, e painéis de reservas, locações, financeiro/cobranças, emergências, viagens em andamento e oficina. Esse mesmo tempo real **torna a reserva mais simples e objetiva** (necessidade 2 do enunciado): o app exibe disponibilidade e ETA reais para o cliente escolher melhor.

### Motor de cobrança dinâmica

Acionado na devolução; valor final = base ± ajustes por km, tempo, consumo, infrações e score de condução. É idempotente (evita duplicidade) e auditável por time travel. **Valor: ↑ receita, ↓ contestações.**

### Apoio a emergências (hot path)

Borda detecta → `alerts.critical` → Spark orquestra → resposta (guincho, seguradora, substituto), montando em paralelo o **dossiê de sinistro** com todas as informações à regulação. **Valor: ↓ tempo de resposta.**

07

Concierge de viagem por IA

PARTE B

Assistente de voz cujo **backend** combina Processamento de Linguagem Natural e **RAG** sobre um banco vetorial de POIs, sem chave de LLM paga. O RAG dá **proveniência auditável** e reduz alucinação — a mesma base sustenta os dossiês de emergência.

[Voz] > ASR > [PLN + agente + function calling] > banco vetorial (RAG) > POIs + agenda > TTS + GPS

EXEMPLO

"Após a reunião, quero almoçar em restaurante típico bem avaliado com estacionamento fácil e, à tarde, ver arte nos meus gostos." → o agente lê a localização, consulta POIs por similaridade e a agenda, monta o roteiro e **injeta os waypoints no GPS**. Privacidade: consentimento, PII, wake-word on-device.

08

Casos de uso e rastreabilidade das entidades

Três casos reais — crítico, usual e analítico — mostram onde cada entidade modelada aparece; dois apontam para a extensão (+).

| Entidade (+ = extensão)  | 1 · Colisão (crítico) | 2 · Cobrança (usual)    | 3 · Frota (analítico)      |
|--------------------------|-----------------------|-------------------------|----------------------------|
| Fato_Sinistro +          | dossiê + resposta     | —                       | —                          |
| Fato_Cobranca +          | recálculo da locação  | valor ± acréscimos      | —                          |
| Fato_Telemetria_Diaria + | buffer pré-impacto    | km, velocidade, eventos | base do score / manutenção |
| Fato_Manutencao +        | —                     | —                       | prob. de falha → agenda    |
| Fato_Movimentacao        | despacho ao pátio     | —                       | Markov → reposicionamento  |
| Dim_Sensor + (SCD-2)     | firmware na hora      | —                       | saúde do sensor            |
| Dim_FaixaConducao +      | —                     | Agressivo → tarifa      | perfil por veículo         |
| Dim_Tempo_Detalhe +      | minuto 18:23          | janelas da viagem       | faixa horária / pico       |

## 09 Custos, nuvem e prova de conceito

BANDA / EGRESS

### Filtrar na borda

Não enviar vídeo bruto corta o maior custo recorrente.

COMPUTE

### Elasticidade

Autoscaling + spot/preemptível; clusters efêmeros no batch.

STORAGE

### Tiering

Quente (Redis/Cassandra) vs. frio (Delta/MinIO), TTL, Bronze→Gold.

OPERAÇÃO

### Gerenciado vs. próprio

Serviços gerenciados reduzem custo; trade-off com lock-in.

### PROVA DE CONCEITO EXECUTÁVEL

O sistema foi implementado em **docker-compose** (Kafka, Flink, Spark, Cassandra, MongoDB, Redis, MinIO, PostgreSQL Gold, Streamlit), com **115 testes unitários** verdes e o DDL do DW estendido validado. Experimentação em *free tiers*: Dataproc/Flink Playground, Databricks, Atlas, Astra, Redis, DynamoDB.

## 10 Alternativas descartadas

| Opção descartada          | Por que não                                                                      |
|---------------------------|----------------------------------------------------------------------------------|
| Somente Data Warehouse    | Batch e schema-on-write não dão tempo real nem absorvem sensores de alto volume. |
| Somente batch             | Emergências e roteirização exigem segundos/milissegundos.                        |
| Arquitetura Lambda        | Base de código dupla e risco de divergência; o problema é streaming-first.       |
| Banco relacional único    | Não escala a escrita massiva nem entrega cache sub-ms (CAP).                     |
| Vídeo bruto pela rede     | Banda cara e finita; fica na borda.                                              |
| Hadoop MapReduce clássico | Alta latência de I/O; o Spark (DAG, in-memory) o substitui.                      |

---

## 11 Conclusão

A solução proposta é uma arquitetura **resiliente, escalável e em tempo real**, orientada a **valor**: filtra na borda, isola fluxos no Kafka, adota **Kappa + Delta Lake**, processa em três regimes (Spark/Flink), persiste de forma poliglota e serve dashboard, cobrança auditável, dossiês e concierge de IA. Ela **envolve e estende o DW** da Avaliação 02 como camada Gold e foi materializada num sistema executável validado, mantendo o custo de nuvem sob controle e movendo os indicadores de negócio do consórcio.

Fundamentação teórica completa, metodologia detalhada e referências (ABNT): ver o *relatório acadêmico* que acompanha esta proposta.